

WEEK 8: EXERCISE 8

Database System and Cloud (BERR2243)



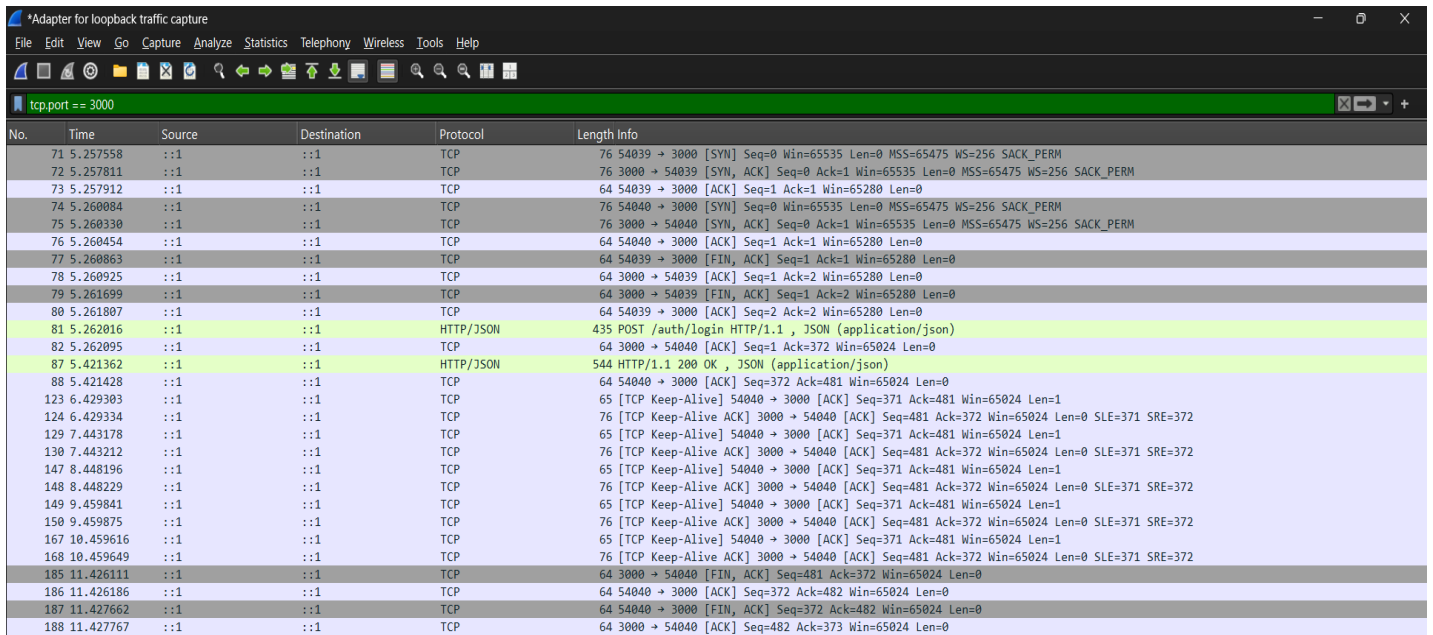
Provided by:

Group Member	Id No.
Ahmad Muhaimin Azeem Bin Adri Wafik	B122310496, 2 BERR S2
Syafiq Iman Bin Suhaimi	B122310218, 2 BERR S2
Ery Fahrhan Bin Mohamed Ali	B122310448, 2 BERR S2

A) CAPTURING AUTHENTICATION FLOW

1. Capturing Log In Request:

Capturing Protected Resource Access was done when a user logs in account in Postman. Below is screenshot of its file. (login.pcapng)

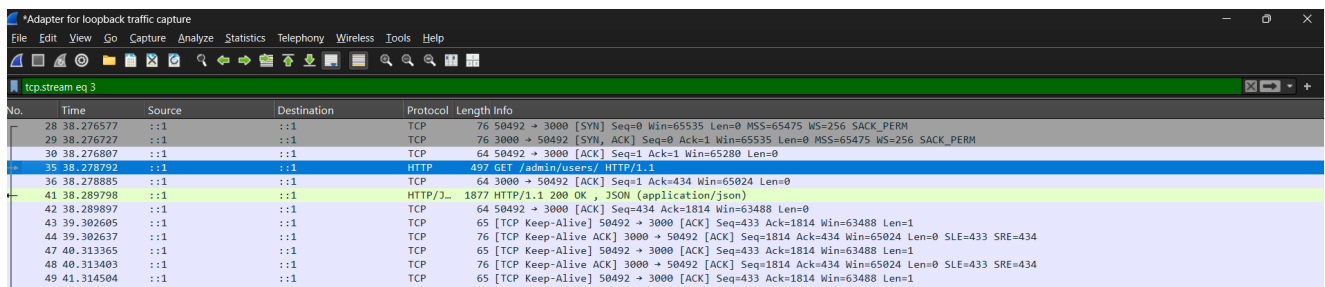


The screenshot shows a Wireshark packet capture for the file 'login.pcapng'. The filter is 'tcp.port == 3000'. The packet list shows a series of TCP connections and an HTTP POST request. The packet details pane shows the structure of the POST request, including the 'Content-Type' and 'Content-Length' fields.

No.	Time	Source	Destination	Protocol	Length	Info
71	5.257558	:::1	:::1	TCP	76	54039 → 3000 [SYN] Seq=0 Win=65535 Len=0 MSS=65475 WS=256 SACK_PERM
72	5.257811	:::1	:::1	TCP	76	3000 → 54039 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=65475 WS=256 SACK_PERM
73	5.257912	:::1	:::1	TCP	64	54039 → 3000 [ACK] Seq=1 Ack=1 Win=65280 Len=0
74	5.260084	:::1	:::1	TCP	76	54040 → 3000 [SYN] Seq=0 Win=65535 Len=0 MSS=65475 WS=256 SACK_PERM
75	5.260330	:::1	:::1	TCP	76	3000 → 54040 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=65475 WS=256 SACK_PERM
76	5.260454	:::1	:::1	TCP	64	54040 → 3000 [ACK] Seq=1 Ack=1 Win=65280 Len=0
77	5.260863	:::1	:::1	TCP	64	54039 → 3000 [FIN, ACK] Seq=1 Ack=1 Win=65280 Len=0
78	5.260925	:::1	:::1	TCP	64	3000 → 54039 [ACK] Seq=1 Ack=2 Win=65280 Len=0
79	5.261699	:::1	:::1	TCP	64	3000 → 54039 [FIN, ACK] Seq=1 Ack=2 Win=65280 Len=0
80	5.261807	:::1	:::1	TCP	64	54039 → 3000 [ACK] Seq=2 Ack=2 Win=65280 Len=0
81	5.262016	:::1	:::1	HTTP/JSON	435	POST /auth/login HTTP/1.1 , JSON (application/json)
82	5.262095	:::1	:::1	TCP	64	3000 → 54040 [ACK] Seq=1 Ack=372 Win=65024 Len=0
87	5.421362	:::1	:::1	HTTP/JSON	544	HTTP/1.1 200 OK , JSON (application/json)
88	5.421428	:::1	:::1	TCP	64	54040 → 3000 [ACK] Seq=372 Ack=481 Win=65024 Len=0
123	6.429303	:::1	:::1	TCP	65	[TCP Keep-Alive] 54040 → 3000 [ACK] Seq=371 Ack=481 Win=65024 Len=1
124	6.429334	:::1	:::1	TCP	76	[TCP Keep-Alive ACK] 3000 → 54040 [ACK] Seq=481 Ack=372 Win=65024 Len=0 SLE=371 SRE=372
129	7.443178	:::1	:::1	TCP	65	[TCP Keep-Alive] 54040 → 3000 [ACK] Seq=371 Ack=481 Win=65024 Len=1
130	7.443212	:::1	:::1	TCP	76	[TCP Keep-Alive ACK] 3000 → 54040 [ACK] Seq=481 Ack=372 Win=65024 Len=0 SLE=371 SRE=372
147	8.448196	:::1	:::1	TCP	65	[TCP Keep-Alive] 54040 → 3000 [ACK] Seq=371 Ack=481 Win=65024 Len=1
148	8.448229	:::1	:::1	TCP	76	[TCP Keep-Alive ACK] 3000 → 54040 [ACK] Seq=481 Ack=372 Win=65024 Len=0 SLE=371 SRE=372
149	9.459841	:::1	:::1	TCP	65	[TCP Keep-Alive] 54040 → 3000 [ACK] Seq=371 Ack=481 Win=65024 Len=1
150	9.459875	:::1	:::1	TCP	76	[TCP Keep-Alive ACK] 3000 → 54040 [ACK] Seq=481 Ack=372 Win=65024 Len=0 SLE=371 SRE=372
167	10.459616	:::1	:::1	TCP	65	[TCP Keep-Alive] 54040 → 3000 [ACK] Seq=371 Ack=481 Win=65024 Len=1
168	10.459649	:::1	:::1	TCP	76	[TCP Keep-Alive ACK] 3000 → 54040 [ACK] Seq=481 Ack=372 Win=65024 Len=0 SLE=371 SRE=372
185	11.426111	:::1	:::1	TCP	64	3000 → 54040 [FIN, ACK] Seq=481 Ack=372 Win=65024 Len=0
186	11.426186	:::1	:::1	TCP	64	54040 → 3000 [ACK] Seq=372 Ack=482 Win=65024 Len=0
187	11.427662	:::1	:::1	TCP	64	54040 → 3000 [FIN, ACK] Seq=372 Ack=482 Win=65024 Len=0
188	11.427767	:::1	:::1	TCP	64	3000 → 54040 [ACK] Seq=482 Ack=373 Win=65024 Len=0

2. Capture Protected Resource Access

Capturing Protected Resource Access was done when a user logs in account in Postman. Below is screenshot of its file (admin-access.pcapng)

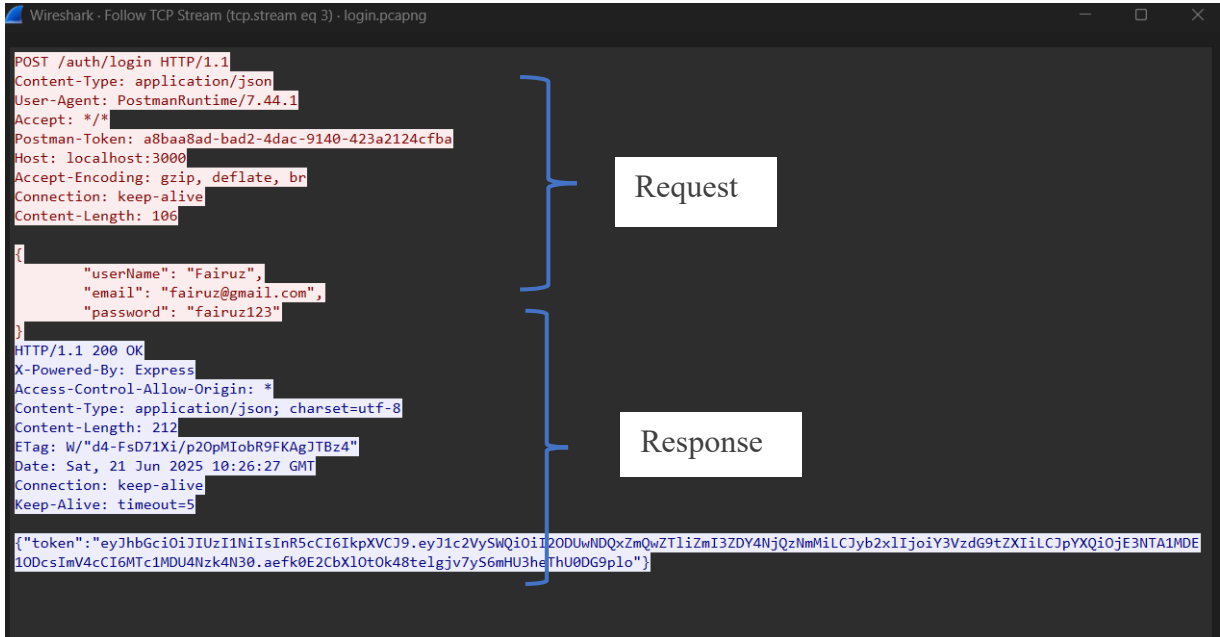


The screenshot shows a Wireshark packet capture for the file 'admin-access.pcapng'. The filter is 'tcp.stream eq 3'. The packet list shows a series of TCP connections and an HTTP GET request. The packet details pane shows the structure of the GET request, including the 'Content-Type' and 'Content-Length' fields.

No.	Time	Source	Destination	Protocol	Length	Info
28	38.276577	:::1	:::1	TCP	76	50492 → 3000 [SYN] Seq=0 Win=65535 Len=0 MSS=65475 WS=256 SACK_PERM
29	38.276727	:::1	:::1	TCP	76	3000 → 50492 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=65475 WS=256 SACK_PERM
30	38.276807	:::1	:::1	TCP	64	50492 → 3000 [ACK] Seq=1 Ack=1 Win=65280 Len=0
35	38.278792	:::1	:::1	HTTP	497	GET /admin/users/ HTTP/1.1
36	38.278885	:::1	:::1	TCP	64	3000 → 50492 [ACK] Seq=1 Ack=434 Win=65024 Len=0
41	38.289798	:::1	:::1	HTTP/JSON	1877	HTTP/1.1 200 OK , JSON (application/json)
42	38.289897	:::1	:::1	TCP	64	50492 → 3000 [ACK] Seq=434 Ack=1814 Win=63488 Len=0
43	39.302605	:::1	:::1	TCP	65	[TCP Keep-Alive] 50492 → 3000 [ACK] Seq=433 Ack=1814 Win=63488 Len=1
44	39.302637	:::1	:::1	TCP	76	[TCP Keep-Alive ACK] 3000 → 50492 [ACK] Seq=1814 Ack=434 Win=65024 Len=0 SLE=433 SRE=434
47	40.313365	:::1	:::1	TCP	65	[TCP Keep-Alive] 50492 → 3000 [ACK] Seq=433 Ack=1814 Win=63488 Len=1
48	40.313403	:::1	:::1	TCP	76	[TCP Keep-Alive ACK] 3000 → 50492 [ACK] Seq=1814 Ack=434 Win=65024 Len=0 SLE=433 SRE=434
49	41.314504	:::1	:::1	TCP	65	[TCP Keep-Alive] 50492 → 3000 [ACK] Seq=433 Ack=1814 Win=63488 Len=1

B) TCP Stream Analysis

1. Analyze Login Request

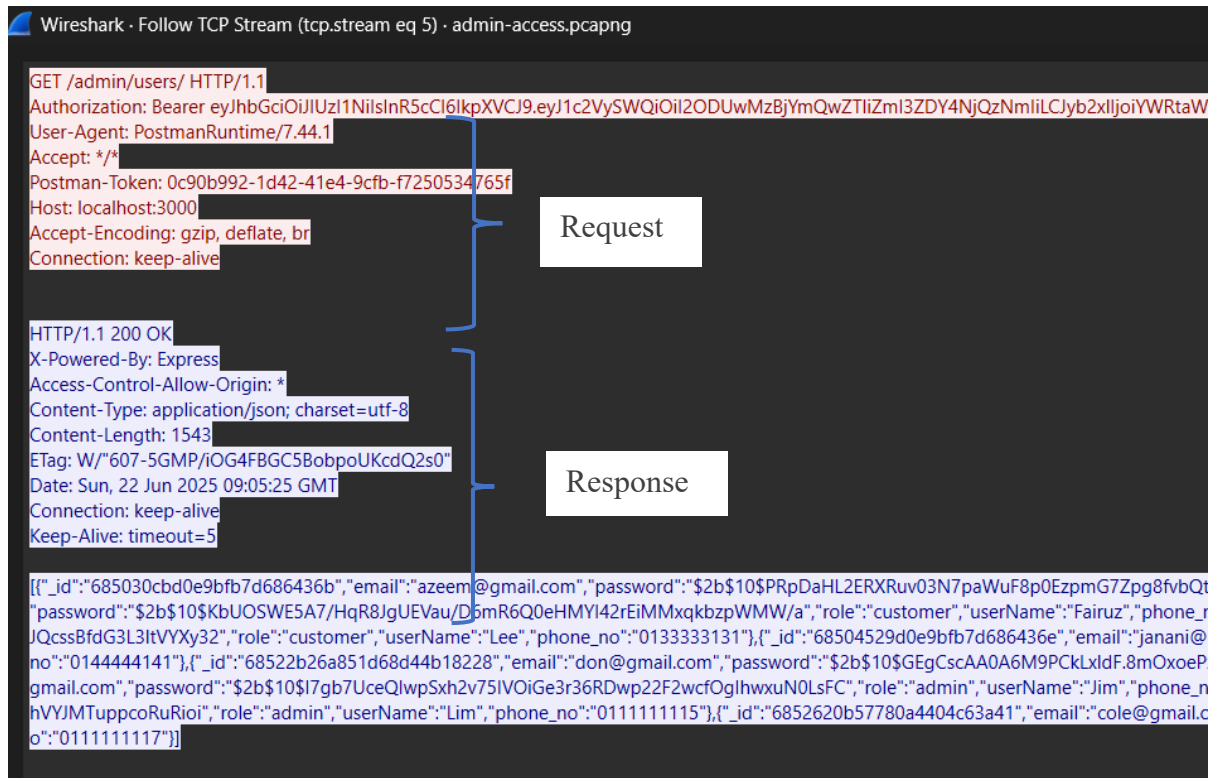


Request Analysis: The login request was a POST request sent to /login. It included the correct Content-Type: application/json header, and the body contained the correct username and password. No token was included in the request as expected.

Response Analysis: The server returned a 200 OK status, indicating a successful login, and the response body included a valid JWT token in the "token" field. This token will be used for subsequent authenticated requests.

Security Analysis: If this traffic were sent over HTTP (instead of HTTPS), the login credentials and JWT token could be intercepted by an attacker. It's critical to use HTTPS to ensure security.

2. Analyze Protected Request



Request Analysis: The HTTP request sent to /admin/users contained a valid JWT token in the Authorization header (Bearer <token>). The token was visible in the request, which poses a security risk if the traffic was unencrypted (over HTTP instead of HTTPS).

Response Analysis: The server returned a 200 OK status with a JSON body containing a list of users. The response confirms the user was authenticated successfully, and sensitive data was returned. This could be a security concern if intercepted by a malicious threats.

EXERCISE QUESTIONS

1. Protocol Analysis:

- i. What is the exact sequence of TCP packets during the 3-way handshake?

*Adapter for loopback traffic capture					
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help					
tcp.port == 3000					
No.	Time	Source	Destination	Protocol	Length Info
71	5.257558	:::1	:::1	TCP	76 54039 → 3000 [SYN] Seq=0 Win=65535 Len=0 MSS=65475 WS=256 SACK_PERM
72	5.257811	:::1	:::1	TCP	76 3000 → 54039 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=65475 WS=256 SACK_PERM
73	5.257912	:::1	:::1	TCP	64 54039 → 3000 [ACK] Seq=1 Ack=1 Win=65280 Len=0

Step	Direction	Description
1	Client → Server	SYN: Client requests connection.
2	Server → Client	SYN-ACK: Server acknowledges and requests.
3	Client → Server	ACK: Client acknowledges server's request.

2. How many packets are exchanged for a successful login request?

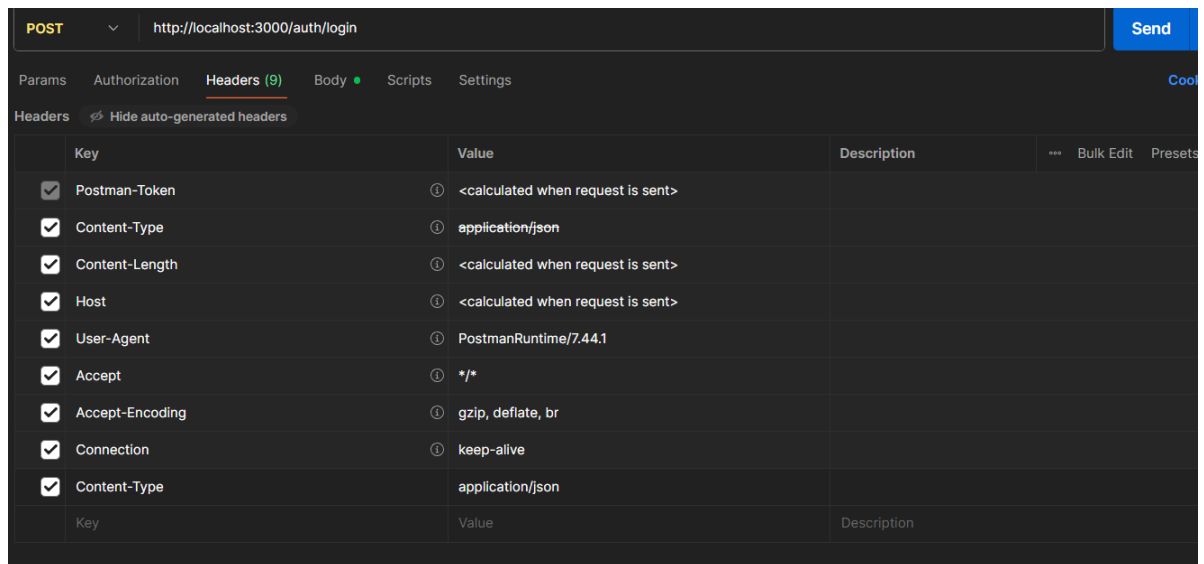
22	23.576337	:::1	:::1	TCP	76 50327 → 3000 [SYN] Seq=0 Win=65535 Len=0 MSS=65475 WS=256 SACK_PERM
23	23.576475	:::1	:::1	TCP	76 3000 → 50327 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=65475 WS=256 SACK_PERM
24	23.576548	:::1	:::1	TCP	64 50327 → 3000 [ACK] Seq=1 Ack=1 Win=65280 Len=0
25	23.577311	:::1	:::1	HTTP/JSON	435 POST /auth/login HTTP/1.1 , JSON (application/json)
30	23.577872	:::1	:::1	TCP	64 3000 → 50327 [ACK] Seq=1 Ack=372 Win=65024 Len=0
35	23.730956	:::1	:::1	HTTP/JSON	544 HTTP/1.1 200 OK , JSON (application/json)
36	23.731055	:::1	:::1	TCP	64 50327 → 3000 [ACK] Seq=372 Ack=481 Win=65024 Len=0
37	24.731833	:::1	:::1	TCP	65 [TCP Keep-Alive] 50327 → 3000 [ACK] Seq=371 Ack=481 Win=65024 Len=1
38	24.731866	:::1	:::1	TCP	76 [TCP Keep-Alive ACK] 3000 → 50327 [ACK] Seq=481 Ack=372 Win=65024 Len=0 SLE=371 SRE=372
39	25.736527	:::1	:::1	TCP	65 [TCP Keep-Alive] 50327 → 3000 [ACK] Seq=371 Ack=481 Win=65024 Len=1
40	25.736559	:::1	:::1	TCP	76 [TCP Keep-Alive ACK] 3000 → 50327 [ACK] Seq=481 Ack=372 Win=65024 Len=0 SLE=371 SRE=372
41	26.748540	:::1	:::1	TCP	65 [TCP Keep-Alive] 50327 → 3000 [ACK] Seq=371 Ack=481 Win=65024 Len=1
42	26.748586	:::1	:::1	TCP	76 [TCP Keep-Alive ACK] 3000 → 50327 [ACK] Seq=481 Ack=372 Win=65024 Len=0 SLE=371 SRE=372
43	27.757875	:::1	:::1	TCP	65 [TCP Keep-Alive] 50327 → 3000 [ACK] Seq=371 Ack=481 Win=65024 Len=1
44	27.757908	:::1	:::1	TCP	76 [TCP Keep-Alive ACK] 3000 → 50327 [ACK] Seq=481 Ack=372 Win=65024 Len=0 SLE=371 SRE=372
45	28.767336	:::1	:::1	TCP	65 [TCP Keep-Alive] 50327 → 3000 [ACK] Seq=371 Ack=481 Win=65024 Len=1
46	28.767371	:::1	:::1	TCP	76 [TCP Keep-Alive ACK] 3000 → 50327 [ACK] Seq=481 Ack=372 Win=65024 Len=0 SLE=371 SRE=372
47	29.735889	:::1	:::1	TCP	64 3000 → 50327 [FIN, ACK] Seq=481 Ack=372 Win=65024 Len=0
48	29.738674	:::1	:::1	TCP	64 50327 → 3000 [ACK] Seq=372 Ack=482 Win=65024 Len=0
49	29.739435	:::1	:::1	TCP	64 50327 → 3000 [FIN, ACK] Seq=372 Ack=482 Win=65024 Len=0
50	29.739521	:::1	:::1	TCP	64 3000 → 50327 [ACK] Seq=482 Ack=373 Win=65024 Len=0

No of packets.	Protocol	Info
1	Initial Packet (SYN)	50327 → 3000 [SYN] Seq=0 Win=65535 Len=0 MSS=65475 WS=256 SACK_PERM
2	SYN-ACK Packet	3000 → 50327 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=65475 WS=256 SACK_PERM
3	TCP Handshake (ACK)	50327 → 3000 [ACK] Seq=1 Ack=1 Win=65280 Len=0
4	HTTP Request (Login)	435 POST /auth/login HTTP/1.1 , JSON (application/json)

5	TCP Handshake (ACK)	3000 → 50327 [ACK] Seq=1 Ack=372 Win=65024 Len=0
6	HTTP Response	544 HTTP/1.1 200 OK , JSON (application/json)

2. Header Inspection:

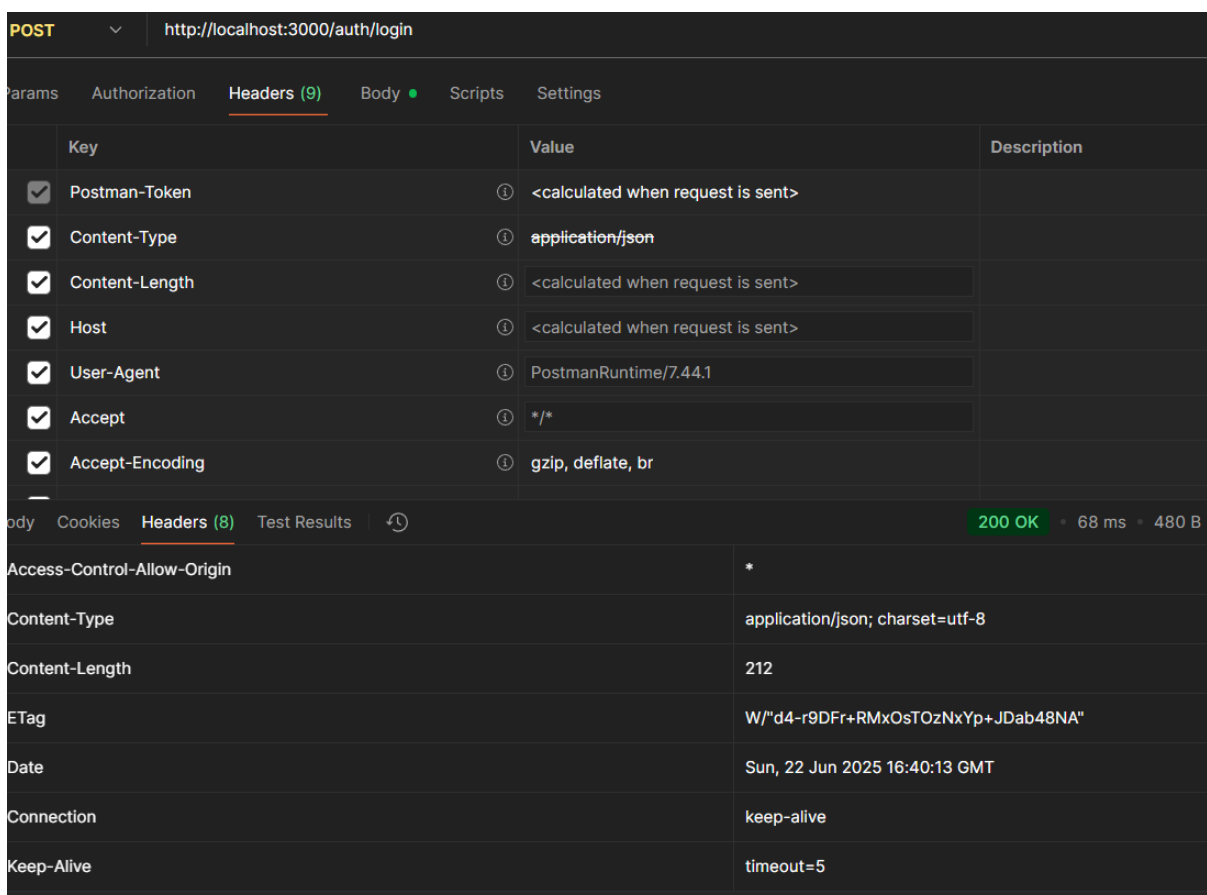
i. What headers does Postman include that a browser might omit?



Key	Value	Description
☒ Postman-Token	<calculated when request is sent>	
☒ Content-Type	application/json	
☒ Content-Length	<calculated when request is sent>	
☒ Host	<calculated when request is sent>	
☒ User-Agent	PostmanRuntime/7.44.1	
☒ Accept	*/*	
☒ Accept-Encoding	gzip, deflate, br	
☒ Connection	keep-alive	
☒ Content-Type	application/json	

ii. How is the Content-Length header calculated?

Option 1: By referring Headers section in Postman



Key	Value	Description
☒ Postman-Token	<calculated when request is sent>	
☒ Content-Type	application/json	
☒ Content-Length	<calculated when request is sent>	
☒ Host	<calculated when request is sent>	
☒ User-Agent	PostmanRuntime/7.44.1	
☒ Accept	*/*	
☒ Accept-Encoding	gzip, deflate, br	

Key	Value	Description
Access-Control-Allow-Origin	*	
Content-Type	application/json; charset=utf-8	
Content-Length	212	
ETag	W/"d4-r9DFr+RMxOsToZNxYp+JDab48NA"	
Date	Sun, 22 Jun 2025 16:40:13 GMT	
Connection	keep-alive	
Keep-Alive	timeout=5	

Option 2: By referring Follow TCP Stream in WireShark

```
Wireshark · Follow TCP Stream (tcp.stream eq 3) · login.pcapng

POST /auth/login HTTP/1.1
Content-Type: application/json
User-Agent: PostmanRuntime/7.44.1
Accept: */*
Postman-Token: a8baa8ad-bad2-4dac-9140-423a2124cfba
Host: localhost:3000
Accept-Encoding: gzip, deflate, br
Connection: keep-alive
Content-Length: 106

{
  "userName": "Fairuz",
  "email": "fairuz@gmail.com",
  "password": "fairuz123"
}

HTTP/1.1 200 OK
X-Powered-By: Express
Access-Control-Allow-Origin: *
Content-Type: application/json; charset=utf-8
Content-Length: 212
ETag: W/"d4-FsD71Xi/p20pMIobR9FKAgJTbz4"
Date: Sat, 21 Jun 2025 10:26:27 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySWQiOiI2ODUwNDQxZmQwZTliZmI3ZDY4IiwiaWF0IjoiMTY1ODUwNDQxZmQwZTliZmI3ZDY4IiwiaXNjaWkiOiJ1c2VySWQi"}
}
```

3. JWT Transmission:

i. At which OSI layer is the JWT token visible? Why is this dangerous?

The JWT token is visible at the Application Layer (Layer 7) of the OSI model.

Reason: JWT (JSON Web Token) is part of the HTTP request/response body and is transmitted as part of the Authorization header in HTTP requests (e.g., Authorization: Bearer <JWT_token>). Since HTTP operates at the Application Layer, the JWT token is handled at this layer, alongside other HTTP-based protocols like REST and WebSocket.

```
POST /auth/login HTTP/1.1
Content-Type: application/json
User-Agent: PostmanRuntime/7.44.1
Accept: */*
Postman-Token: 2b8b857d-170a-48bf-b6a4-22159901bb4c
Host: localhost:3000
Accept-Encoding: gzip, deflate, br
Connection: keep-alive
Content-Length: 106

{
  "userName": "Janani",
  "email": "janani@gmail.com",
  "password": "janani123"
}

HTTP/1.1 200 OK
X-Powered-By: Express
Access-Control-Allow-Origin: *
Content-Type: application/json; charset=utf-8
Content-Length: 212
ETag: W/"d4-jJwGQk2+qXeKincmV/kQx8eZbUA"
Date: Mon, 23 Jun 2025 07:30:03 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySWQiOiI2ODUwNDUyOWQwZTliZmI3ZDY4IiwiaWF0IjoiMTY1ODUwNDUyOWQwZTliZmI3ZDY4IiwiaXNjaWkiOiJ1c2VySWQi"}
}
```

ii. How would HTTPS change what you see in Wireshark?

With HTTP (non-encrypted):

The data (including sensitive information like JWT tokens) is sent as plaintext.

You can easily see the contents of HTTP headers, including the Authorization: Bearer <JWT_token>, and the full request/response body.

With HTTPS (encrypted):

1131	5.267559	2401:e120:2034:d9:5...	2404:6800:4001:803:...	TLSv1.3	1905	Client Hello (SNI=yt3.googleusercontent.com)
1137	5.274220	2401:e120:2034:d9:5...	2404:6800:4001:80a:...	TLSv1.3	1831	Client Hello (SNI=www.gstatic.com)
1138	5.274818	2401:e120:2034:d9:5...	2404:6800:4001:80a:...	TLSv1.3	1863	Client Hello (SNI=www.gstatic.com)
1142	5.281094	2401:e120:2034:d9:5...	2404:6800:4001:803:...	TLSv1.3	1841	Client Hello (SNI=yt3.googleusercontent.com)
1150	5.411802	2404:6800:4001:80a:...	2401:e120:2034:d9:5...	QUIC	1292	Initial, SCID=ed2f2bdc4f813fa0, PKN: 3, CRYPTO, PADDING
1151	5.411802	2404:6800:4001:80a:...	2401:e120:2034:d9:5...	QUIC	1292	Initial, SCID=ed2f2bdc4f813fa0, PKN: 4, CRYPTO, PADDING
1153	5.411802	2404:6800:4001:803:...	2401:e120:2034:d9:5...	QUIC	1292	Initial, SCID=f70ed9a8d41d3e1c, PKN: 3, CRYPTO, PADDING
1154	5.411802	2404:6800:4001:803:...	2401:e120:2034:d9:5...	QUIC	1292	Initial, SCID=f70ed9a8d41d3e1c, PKN: 4, CRYPTO, PADDING
1172	5.590804	2404:6800:4001:803:...	2401:e120:2034:d9:5...	TLSv1.3	1506	Server Hello, Change Cipher Spec
1182	5.593461	2404:6800:4001:803:...	2401:e120:2034:d9:5...	TLSv1.3	541	Application Data
1186	5.593461	2404:6800:4001:80a:...	2401:e120:2034:d9:5...	TLSv1.3	4370	Server Hello, Change Cipher Spec
1187	5.593461	2404:6800:4001:80a:...	2401:e120:2034:d9:5...	TLSv1.3	4370	Server Hello, Change Cipher Spec

The entire communication (headers, body, etc.) between the client and server is encrypted.

Wireshark cannot directly see the HTTP request/response content, including the JWT token, because the data is ciphertext.

4. Error Handling:

i. Capture an invalid login attempt. What status code is returned?

The status code for an invalid login attempt is typically 401 Unauthorized. This indicates that the credentials provided are incorrect and the server is refusing to authorize the request.

```
POST /auth/login HTTP/1.1
Content-Type: application/json
User-Agent: PostmanRuntime/7.44.1
Accept: */*
Postman-Token: 1ae42583-80b9-46e6-a391-2bef36d9b286
Host: localhost:3000
Accept-Encoding: gzip, deflate, br
Connection: keep-alive
Content-Length: 103

{
  "userName": "Aaron",
  "email": "aaron@gmail.com",
  "password": "aaron123"
}
HTTP/1.1 401 Unauthorized
X-Powered-By: Express
Access-Control-Allow-Origin: *
Content-Type: application/json; charset=utf-8
Content-Length: 31
ETag: W/"1f-mRYQ6Yx/raK/ssDeWseqQCih0yM"
Date: Mon, 23 Jun 2025 08:21:31 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"error":"Invalid credentials"}
```


ii. How does the TCP stream differ for a 401 vs 200 response?

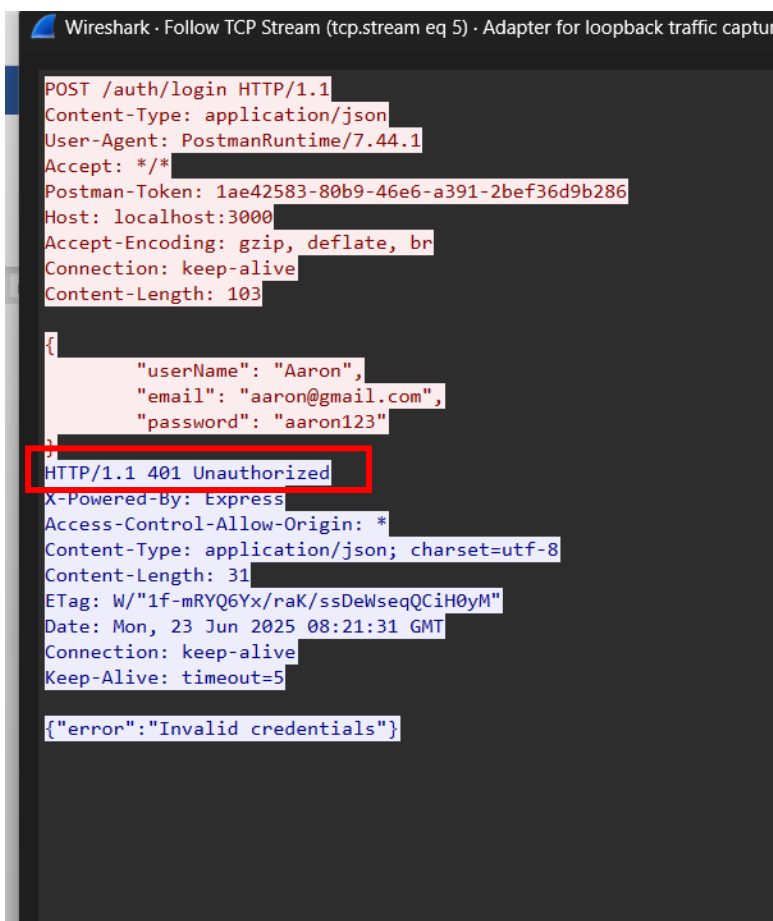
401 Code:

The request requires authentication, but the credentials provided were incorrect.

TCP Stream for 401 Unauthorized:

HTTP Headers: The server responds with HTTP/1.1 401 Unauthorized.

Body: There may or may not be content in the body, but the status is mainly used to signal that authorization failed.

A screenshot of the Wireshark network protocol analyzer. The top bar shows 'Wireshark · Follow TCP Stream (tcp.stream eq 5) · Adapter for loopback traffic capture'. The main pane displays the details of an HTTP request and response. The request is a POST to '/auth/login' with headers: 'Content-Type: application/json', 'User-Agent: PostmanRuntime/7.44.1', 'Accept: */*', 'Postman-Token: 1ae42583-80b9-46e6-a391-2bef36d9b286', 'Host: localhost:3000', 'Accept-Encoding: gzip, deflate, br', 'Connection: keep-alive', and 'Content-Length: 103'. The body is a JSON object: {'userName': 'Aaron', 'email': 'aaron@gmail.com', 'password': 'aaron123'}. The response is an HTTP/1.1 401 Unauthorized status, highlighted with a red box. Its headers include: 'X-Powered-By: Express', 'Access-Control-Allow-Origin: *', 'Content-Type: application/json; charset=utf-8', 'Content-Length: 31', 'ETag: W/"1f-mRYQ6Yx/raK/ssDeWseqQCih0yM"', 'Date: Mon, 23 Jun 2025 08:21:31 GMT', 'Connection: keep-alive', and 'Keep-Alive: timeout=5'. The body is a JSON object: {'error': 'Invalid credentials'}.

```
POST /auth/login HTTP/1.1
Content-Type: application/json
User-Agent: PostmanRuntime/7.44.1
Accept: */*
Postman-Token: 1ae42583-80b9-46e6-a391-2bef36d9b286
Host: localhost:3000
Accept-Encoding: gzip, deflate, br
Connection: keep-alive
Content-Length: 103

{
  "userName": "Aaron",
  "email": "aaron@gmail.com",
  "password": "aaron123"
}

HTTP/1.1 401 Unauthorized
X-Powered-By: Express
Access-Control-Allow-Origin: *
Content-Type: application/json; charset=utf-8
Content-Length: 31
ETag: W/"1f-mRYQ6Yx/raK/ssDeWseqQCih0yM"
Date: Mon, 23 Jun 2025 08:21:31 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"error":"Invalid credentials"}
```

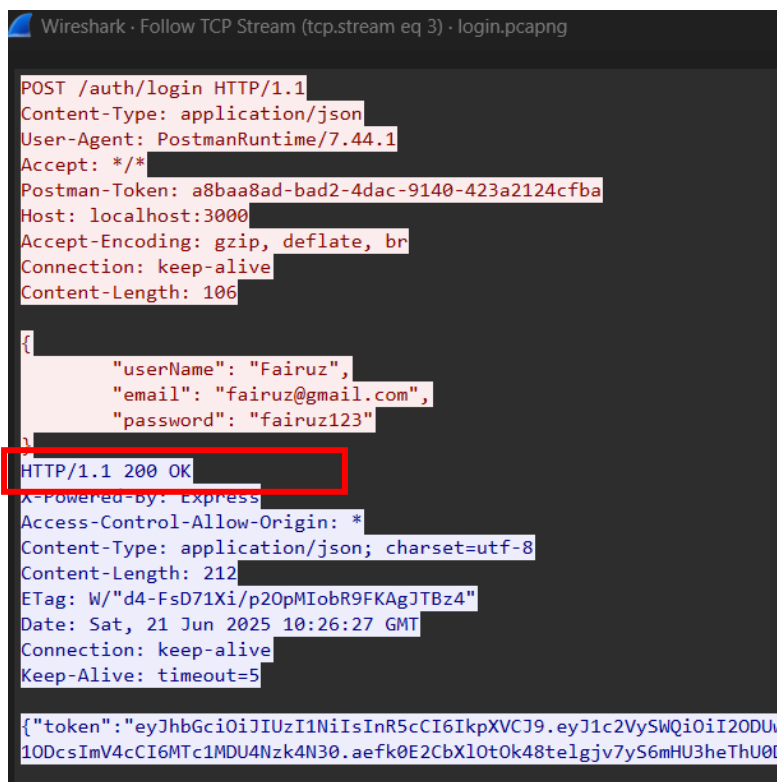
200 OK:

This is a successful response indicating that the request was valid and the server has processed it successfully.

TCP Stream for 200 OK:

HTTP Headers: The server responds with HTTP/1.1 200 OK in the header.

Body: The response body will usually contain the requested data (e.g., JSON, HTML, etc).



The image shows a Wireshark packet capture window titled "Follow TCP Stream (tcp.stream eq 3) - login.pcapng". It displays the details of an HTTP transaction. The request is a POST to /auth/login with headers: Content-Type: application/json, User-Agent: PostmanRuntime/7.44.1, Accept: */*, Postman-Token: a8baa8ad-bad2-4dac-9140-423a2124cfba, Host: localhost:3000, Accept-Encoding: gzip, deflate, br, Connection: keep-alive, and Content-Length: 106. The request body is a JSON object: {"userName": "Fairuz", "email": "fairuz@gmail.com", "password": "fairuz123"}. The response is an HTTP/1.1 200 OK with headers: X-Powered-by: Express, Access-Control-Allow-Origin: *, Content-Type: application/json; charset=utf-8, Content-Length: 212, ETag: W/"d4-FsD71Xi/p2OpMIobR9FKAgJTBz4", Date: Sat, 21 Jun 2025 10:26:27 GMT, Connection: keep-alive, and Keep-Alive: timeout=5. The response body is a JSON object: {"token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySWQiOiI2ODUv10DcsImV4cCI6MTc1MDU4Nzk4N30.aefk0E2CbXl0tOk48telgfv7yS6mHU3heThU0t".

5. Performance:

i. Measure time between POST /login request and response.

No.	Time	Source	Destination	Protocol	Length Info
10	6.260297	::1	::1	TCP	76 50957 → 3000 [SYN] Seq=0 Win=65535 Len=0 MSS=65475 WS=256 SACK_PERM
11	6.260376	::1	::1	TCP	76 3000 → 50957 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=65475 WS=256 SACK_PERM
12	6.260414	::1	::1	TCP	64 50957 → 3000 [ACK] Seq=1 Ack=1 Win=65280 Len=0
17	6.261137	::1	::1	HTTP/JSON	435 POST /auth/login HTTP/1.1, JSON (application/json)
18	6.261177	::1	::1	TCP	64 3000 → 50957 [ACK] Seq=1 Ack=372 Win=65024 Len=0
23	6.329504	::1	::1	HTTP/JSON	544 HTTP/1.1 200 OK, JSON (application/json)

No.	Time	Length Info
17	6.261137	435 POST /auth/login HTTP/1.1, JSON (application/ json)

23	6.329504	544 HTTP/1.1 200 OK, JSON (application/json)
----	----------	--

Time login response-request = 6.329504 - 6.261137 = 0.068367

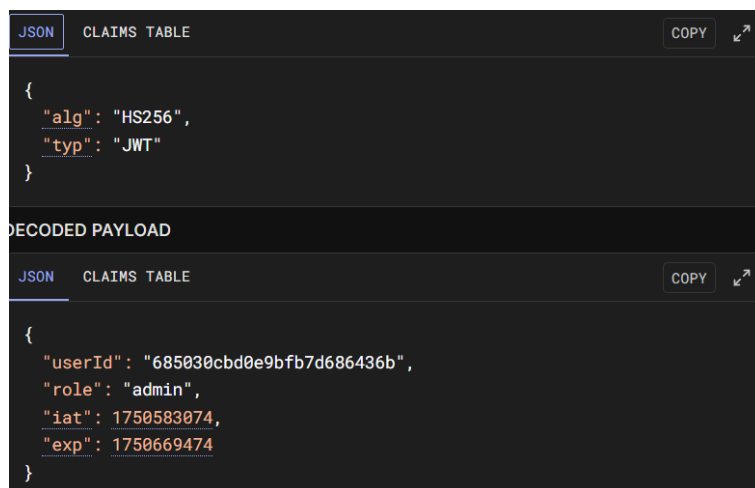
ii. What contributes to this latency?

Contributor	Description
Network Congestion	High traffic on a network can cause delays as data packets queue up for transmission.
Bandwidth Limitations	Limited bandwidth can cause delays in data transmission, particularly in data-intensive applications.
Geographical Distance	Data traveling long distances between distributed nodes can increase latency due to the inherent delays in transmission.
Server Load	Overloaded servers can take longer to process requests, contributing to high latency.
Latency in Database Queries	Complex or inefficient database queries can significantly increase response times.

reference: <https://www.geeksforgeeks.org/system-design/latency-in-system-design/>

6. Security Exercise:

i. What user information is exposed?



User Information Exposed:

1userId: "685030cbd0e9bfb7d686436b"

- This is the unique identifier for the user in the system. It's typically used to track the user across different sessions or actions.

role: "admin"

- This indicates the user's **role** in the application, which is admin in this case. The role defines the level of access or permissions the user has (e.g., admin, user, etc.).
- This indicates the user's role in the application, which is admin in this case. The role defines the level of access or permissions the user has (e.g., admin, user, etc.).

iat (Issued At): 1750583074

- This is the timestamp when the JWT was issued (in Unix time). The value 1750583074 corresponds to Wednesday, March 23, 2025 1:04:34 PM UTC.

exp (Expiration): 1750669474

- This is the expiration timestamp of the token, which tells when the token will expire. The value 1750669474 corresponds to Wednesday, March 23, 2025 7:04:34 PM UTC.

ii. Why doesn't this work in production?

Original JWT (as image below):

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySWQiOiI2ODUyMmIyNmE4NTFkNjhhk



Edited JWT key:

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySWQiOiI2ODUyMmIyNmE4NTFkNjhhNDRiMTgyMjgiLCJyb2x1IjoieY3VzdG9tZXIiLCJpYXQiOiE3NTA2ODY1NjAsImV4cCI6MTc1MDc3Mjk2MH0.SXRD8Maipk7AtNz0pnihp6sH7pB3WQHwLqbi_6Ng



In production, when the server receives a JWT, it uses the **secret key** to verify that the signature matches the payload and header. Therefore, changing the role from admin to customer, the server will **not match the signature** and will reject the token as **tampered**.

7. Security Recommendations

i. 3 risks of unencrypted JWT transmission

- **JWT Payloads Are Not Encrypted:** JWTs are not encrypted by default. The payload is encoded in Base64URL format, which anyone can decode
- **JWTs Can Lead to Stale Data Issues:** Since JWTs are issued with specific claims, any changes to user roles or permissions after token issuance won't be reflected until the token expires, potentially granting outdated access rights.
- **Weak Secrets Make JWTs Easy to Crack:** Using weak or easily guessable secrets to sign JWTs can make them susceptible to brute-force attacks, allowing attackers to forge valid tokens.

reference: <https://lakin-mohapatra.medium.com/why-jwts-can-be-risky-ad367b2f264f>

ii. 2 ways to enhance API security

- **Use a Centralized Authorization Server**

Instead of letting each API handle security on its own, use one central server to manage things like user logins and permissions. This makes it easier to secure your APIs and manage tokens.

- **Use Different Tokens for Internal and External Use**

For internal API communication, use JWTs (JSON Web Tokens) since they're easy to use. But for external users, use opaque tokens that don't expose sensitive information. This adds an extra layer of security.

reference: <https://curity.io/resources/learn/api-security-best-practices/>

Github link: <https://github.com/Ahmad-Azeem/min-iman/tree/week8>